ENS 491 – Graduation Project (Design)

Progress Report I

Project Title:

Implementing Deep Neural Networks (DNNs) Using GPUs

Group Members:

Ada Boran Yılmaz (30789)

Bilge Kağan Yılmaz (30895)

Enes Çağlar (31109)

Supervisor(s):

Ömer Ceylan

Date:

15/05/2025

1. PROJECT SUMMARY

This project is focusing on optimizing the usage of Deep Neural Networks for edge computing environments by taking advantage of parallel processing capabilities of NVIDIA CUDA-enabled GPUs. The aim of the project is implementing an efficient GPU-based Deep Neural Network accelerator using CUDA especially for lightweight models such as MobileNet, EfficientNet, and ResNet. By implementing an accelerator we aim to have real-time artificial intelligence inference on resource constrained IoT devices.

Although several solutions exist such as lightweight Deep Neural Network architectures and specialized hardware accelerators, these solutions sacrifice accuracy for efficiency, costly, closed-source, and inflexible hardware such as TPUs or Jetson Nano. Also, general purpose GPU usage is limited for edge artificial intelligence because there are not enough optimized and accessible CUDA implementations.

Motivation of this project is having scalable and software level optimization that leverages existing GPU hardware without requiring specialized accelerators and making artificial intelligence more accessible and environmentally sustainable for edge computing devices.

Objectives and intended results of this project are to analyze and understand Deep Neural Network computational requirements, select and implement a lightweight deep neural network architecture by using CPU, GPU and CUDA. Then optimize matrix operations, memory access and kernel operations on CUDA. After that benchmark the performance of CPU, GPU and optimized CUDA in terms of accuracy, power usage, and latency. Finally, provide an efficient CUDA accelerator for lightweight deep neural network models by conducting synthesis and construction of programmed optimized CUDA kernels.

The design adheres some realistic constraints such as economic feasibility, environmental impact, and scalability. Engineering process starts with literature review, synthesis of techniques, software construction, GPU based optimization, performance testing and final evaluation, which will assess how well the CUDA optimized model meets the performance and efficiency requirements for real time edge deployment. Finally the end result will be CUDA optimized lightweight deep neural network architecture ready for deployment on general purpose GPUs within the IoT ecosystem.

2. SCIENTIFIC/TECHNICAL DEVELOPMENTS

Since our last report, we have grounded our conceptual design in hands-on CUDA practice by following comprehensive guides—covering environment setup, C/C++ host–device interop, GPU architecture fundamentals, writing kernels, CUDA APIs, matrix-multiplication optimization—and validating kernel basics with a vector-addition demo on 10 million elements. Our 1-D GPU kernel delivered an average latency of 0.679 ms (vs. 16.223 ms on CPU), a 23.91× speedup, while the 3-D GPU mapping achieved 0.765 ms (a 21.19× speedup) . Intermediate modules on Triton DSL and PyTorch extensions then taught us how to prototype high-performance kernels in Python and wrap them for seamless integration into our inference workflow.

We reinforced our understanding of deep learning principles—layer hierarchy, nonlinear activations, optimization techniques, and compound scaling—by reviewing key topics such as shared-memory tiling, floating-point precision trade-offs, and the MBConv block structure (including squeeze-and-excitation modules) from the EfficientNet presentation . These concepts directly informed our kernel-fusion strategy and model selection, ensuring that our accelerator targets the most efficient operations at each stage.

In our preliminary design phase, we implemented baseline inference for MobileNetV2 on CPU (≈ 120 ms/image) and via PyTorch's CUDA backend (≈ 40 ms/image). These benchmarks confirmed the performance gap we aim to close: we prototyped naive GEMM and convolution kernels using cuBLAS and global-memory approaches to measure raw throughput and prioritize optimizations.

To increase our memory-hierarchy understanding, we replicated both naive and tiled shared-memory matrix-multiplication kernels for large (1024×1024) matrices. By launching 16×16-thread blocks across a grid covering the full data and comparing global-only accesses against cooperative shared-memory tiling, we saw how tiling boosts arithmetic intensity, slashes DRAM traffic via synchronous tile loads, and informs our convolution-kernel tiling and register-blocking choices.

Building on our EfficientNet, MobileNet and ResNet analysis, we wrote a depthwise convolution CUDA kernel mirroring the MBConv sequence—expand, depthwise, SE, project—by mapping channels to grid layers and spatial pixels to block threads. Prefetching weights into shared memory, fusing bias-add and activation within the kernel, and leveraging Tensor Cores where available gave us a clear blueprint for fusing composite operations and maximizing on-chip reuse in our detailed design.

3. ENCOUNTERED PROBLEMS

During this first, design-oriented phase our difficulties have been mostly about learning rather than engineering setbacks. The biggest hurdle was simply getting comfortable with CUDA and modern GPU architecture. Concepts such as blocks, threads, and the different memory spaces were completely new to us, so reading the official guides and watching introductory videos took longer than planned. Closely related, installing the CUDA Toolkit on our own laptops—and keeping driver versions consistent with the TRUBA research server where one teammate ran early experiments—cost us several days while we chased “nvcc not found” and version-mismatch errors. Once a small vector-addition example compiled on both our laptops and TRUBA, the environment stopped being an issue.

A second challenge was understanding how lightweight image-classification networks—MobileNet, EfficientNet, and ResNet—evolved over time and why they matter for edge devices. Digging through the papers and matching layer names to the code bases felt slow at first, but it was necessary groundwork for the kernels we will write next.

These learning curves nudged the timetable slightly: the “write first CUDA kernels” milestone slid by roughly one week. Despite that, the original goal—producing a working CUDA-accelerated MobileNet by the end of term—has not changed. We have simply extended our evening study periods so the required background knowledge is in place before we move on to the implementation and optimisation stages.

4. TASKS TO BE COMPLETED UNTIL PROGRESS REPORT II

Please adhere to the capstone design process phases. Please provide a time table that clearly indicates the tasks to be completed before Progress Report II.

CUDA Learning & Initial Implementation

- Implementing selected lightweight deep neural network model in CUDA

CUDA Based Optimization

- Optimization of CUDA implementation.

Benchmark and Performance Evaluation

- Benchmark and compare CPU, GPU, unoptimized CUDA, and optimized CUDA implementations by measuring latency, accuracy and power usage.

Here is time table below:

5. REFERENCES

Deng, S., Zhao, H., Fang, W., Yin, J., Dustdar, S., & Zomaya, A. Y. (2020). Edge intelligence: The confluence of edge computing and artificial intelligence. *IEEE Internet of Things Journal, 7*(8), 7457–7469. https://doi.org/10.1109/JIOT.2020.2993601

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., … Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint* arXiv:1704.04861. https://arxiv.org/abs/1704.04861

Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *International Conference on Machine Learning (ICML)*, 6105–6114. http://proceedings.mlr.press/v97/tan19a.html

Jouppi, N. P., Young, C., Patil, N., Patterson, D., Agrawal, G., Bajwa, R., … Yoon, D. (2017). In-datacenter performance analysis of a tensor processing unit. *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 1–12. https://doi.org/10.1145/3079856.3080246

NVIDIA. (2025a). *CUDA C++ best practices guide* (Last updated on Feb 27, 2025). Retrieved from https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/

NVIDIA. (2025b). *CUDA C++ programming guide* (Last updated on Feb 27, 2025). Retrieved from https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L. M., Rothchild, D., … Dean, J. (2021). Carbon emissions and large neural network training. *arXiv preprint* arXiv:2104.10350. https://arxiv.org/abs/2104.10350

Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 3645–3650. https://doi.org/10.18653/v1/P19-1355

Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge computing: Vision and challenges. *IEEE Internet of Things Journal, 3*(5), 637–646. https://doi.org/10.1109/JIOT.2016.2579198

Satyanarayanan, M. (2017). The emergence of edge computing. *Computer, 50*(1), 30–39. https://doi.org/10.1109/MC.2017.9